

Wallet Transaction Service – Implementation Plan

1. Finalize Requirements & Business Rules

First, confirm exactly what the system should do.

- User registration
- User login
- JWT authentication
- Wallet creation/association
- View wallet balance
- Credit wallet
- Debit wallet
- Transaction history
- Validation and error handling
- Wallet ownership/access rules
- Insufficient-balance handling

Output: finalized functional requirements and business rules.

2. Project & Environment Setup

Set up the development environment.

- Java 21
- Spring Boot
- Maven
- IntelliJ IDEA, VS Code
- Git/Git repository
- PostgreSQL
- Postman

- Swagger/OpenAPI
- Docker

Create the basic Spring Boot project and configure:

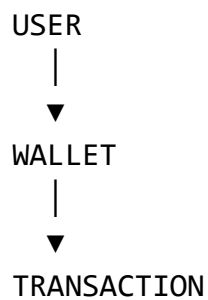
Spring Web
Spring Security
Spring Data JPA
PostgreSQL Driver
Validation
JWT
JUnit
Mockito
Swagger/OpenAPI

Output: running Spring Boot application with database connection.

3. Database Design & Implementation

Design the database based on the requirements.

Initial entities:



Tables:

Users
├── id
├── name
├── email
└── password

Wallets

```
|— id
|— user_id
|— balance
|— status
```

Transactions

```
|— id
|— wallet_id
|— type
|— amount
|— balance_before
|— balance_after
|— status
|— timestamp
```

Also define:

- Primary keys
- Foreign keys
- Constraints
- Relationships
- Indexes where required

Output: database schema/ER diagram and implemented tables.

4. Authentication & Security Implementation

Implement:

Register

↓

Login

↓

Verify credentials

↓
Generate JWT
↓
JWT sent with protected requests
↓
Spring Security validates JWT

Implement:

- Password hashing
- Spring Security configuration
- JWT generation
- JWT validation
- Authentication
- Authorization
- Protected APIs
- Wallet ownership validation

Output: secured APIs.

5. REST API Implementation

Then implement the APIs.

Authentication

POST /auth/register

POST /auth/login

Wallet

GET /wallet/balance

POST /wallet/credit

POST /wallet/debit

Transactions

GET /wallet/transactions

The exact endpoint names can be finalized by your team.

Output: working REST APIs.

6. Business Logic Implementation

This is the important part of the Wallet Service.

For **credit**:

```
Request
↓
Authenticate user
↓
Validate amount
↓
Find wallet
↓
Increase balance
↓
Create transaction record
↓
Return response
```

For **debit**:

```
Request
↓
Authenticate user
↓
Validate amount
↓
Find wallet
↓
```

```
Check sufficient balance
↓
Decrease balance
↓
Create transaction record
↓
Return response
```

Business rules should include things such as:

```
amount > 0
wallet must exist
user must own wallet
debit amount <= available balance
successful operation → transaction record
failed operation → balance must not change
```

7. Transaction Management

This deserves its own implementation step because it is specifically mentioned in your problem.

For example:

```
Debit ₹1,000
|
|— Update wallet balance
|
|— Insert transaction history
```

These operations should be handled atomically using:

@Transactional

So:

```
Both succeed → COMMIT
Either fails → ROLLBACK
```

This protects the consistency of your wallet balance and transaction history.

8. Exception Handling & Validation

Implement common error handling.

For example:

Invalid amount

↓

400 Bad Request

Invalid/expired JWT

↓

401 Unauthorized

Insufficient balance

↓

Business error

Wallet doesn't belong to user

↓

403 Forbidden

Wallet not found

↓

404 Not Found

Use a consistent error response structure.

Output: predictable API error responses.

9. Unit & Integration Testing

Test each important component.

Unit testing

Use:

JUnit 5

Mockito

Test:

- User service
- Wallet service
- Credit
- Debit
- Transaction history
- Validation
- Exception scenarios

Example:

Debit ₹1,000 from ₹5,000
Expected balance = ₹4,000

And:

Debit ₹6,000 from ₹5,000
Expected = failure
Expected balance = ₹5,000

Integration testing

Test:

API → Service → Repository → Database

Also test JWT-protected APIs.

Your requirement mentions **>70% unit-test coverage**, so include coverage measurement in the plan.

10. API Documentation & Testing

Use Swagger/OpenAPI for API documentation.

Use Postman for testing.

Demonstrate:

Register



Login



Receive JWT



Use JWT



Credit



Check balance



Debit



View transaction history

Output: Swagger documentation + Postman test evidence.

11. Dockerization

Once the application works locally:

Spring Boot Application



Dockerfile



Docker Image



Docker Container

Configure the application and database appropriately for the target deployment environment.

Output: Dockerized application with run instructions.

12. AWS Deployment

Then deploy the application to the AWS environment selected by your team/organization.

At this stage:

Dockerized Application



AWS



Running Wallet Transaction Service

Don't specify **EC2/ECS/EKS** in tomorrow's plan unless your team has already decided which one to use.

Output: deployed application and deployment instructions.

13. Documentation & Demo Preparation

Finally prepare:

- Architecture diagram
- API documentation
- Database/ER diagram

- Setup instructions
- API test evidence
- Unit-test results
- Docker instructions
- Deployment instructions
- Known limitations
- Demo flow

Your complete implementation plan

You can put this directly into your document/meeting:

1. Requirement Analysis
↓
2. Environment & Project Setup
↓
3. Database Design & Implementation
↓
4. Authentication & JWT Security
↓
5. REST API Development
↓
6. Business Logic Implementation
↓
7. Transaction Management & Validation
↓
8. Exception Handling
↓
9. Unit & Integration Testing
↓
10. API Documentation & Testing
↓
11. Dockerization
↓
12. AWS Deployment
↓
13. Documentation & Demo

